

Roadmap Substation Maintenance Report API — Langkah Kecil-Kecil

Prinsipnya: tiap checkbox = 1 sesi ngoding (1-2 jam). Selesai satu, centang, lanjut besok. Jangan mikirin langkah ke-15 pas lu masih di langkah ke-2.

FASE 0 — Setup (Hari 1)

- ☐ Bikin project baru di start.spring.io dengan dependency: `Spring Web`, `Spring Data JPA`, `Spring Security`, `PostgreSQL Driver`, `Validation`, `Lombok`
- ☐ Extract & buka di IDE (IntelliJ/VS Code)
- ☐ Jalanin `./mvnw spring-boot:run`, pastiin project nyala tanpa error (walau masih kosong)
- ☐ Push ke GitHub, bikin repo baru namanya `substation-maintenance-api`
- ☐ Bikin file `.gitignore` (Spring Initializr biasanya udah include)

Selesai fase ini kalau: project kosong udah bisa jalan & ke-push ke GitHub.

FASE 1 — Database & Entity Paling Sempel Dulu: Substation (Hari 2-3)

Mulai dari yang PALING gampang dulu — bukan langsung ke Report/User yang kompleks.

- ☐ Bikin entity `Substation` (id, name, code, location) — cuma 4 field, gampang
- ☐ Bikin `SubstationRepository extends JpaRepository`
- ☐ Bikin `SubstationController` dengan 1 endpoint dulu: `POST /api/substations` (create)
- ☐ Test pakai Postman: kirim data, cek masuk ke database gak
- ☐ Kalau udah jalan, tambah endpoint `GET /api/substations` (list semua)
- ☐ Tambah `GET /api/substations/{id}`, `PUT`, `DELETE`

Selesai fase ini kalau: lu bisa CRUD data Substation lewat Postman, dan itu tersimpan permanen di database.

Ini sengaja dibikin paling gampang duluan biar lu ngerasa "progress", sebelum masuk ke bagian yang lebih ribet (auth).

FASE 2 — User & Login Dasar (Hari 4-6)

Ini bagian yang udah pernah lu latihan di project login CRUD sebelumnya — jadi harusnya lebih familiar.

- ☐ Bikin entity `User` (id, name, email, password, role) — role dulu simpan sebagai String aja, belum usah Enum

- ☐ Bikin `UserRepository`
- ☐ Bikin endpoint `POST /api/auth/register` — simpen user baru (password di-hash pakai `BCryptPasswordEncoder`)
- ☐ Test register lewat Postman, cek password di database udah ke-hash (bukan plain text)
- ☐ Setup Spring Security config dasar (izinkan akses `/api/auth/**` tanpa login dulu, sisanya nanti)
- ☐ Bikin endpoint `POST /api/auth/login` — cek email+password, kalau cocok generate JWT
- ☐ Test login, pastiin dapet token JWT balik

Selesai fase ini kalau: lu bisa register user baru dan login dapet token JWT. Belum usah role-based dulu, login aja dulu yang penting jalan.

FASE 3 — JWT Filter & Proteksi Endpoint (Hari 7-9)

Ini bagian yang paling "berat" secara konsep, makanya dipecah lebih kecil lagi:

- ☐ Bikin `JwtUtil` class (generate token, extract username dari token, validasi token) — kalau bingung, cari 1 tutorial JWT Spring Boot yang paling simpel dan ikutin persis dulu, paham logikanya belakangan
- ☐ Bikin `JwtAuthFilter extends OncePerRequestFilter`
- ☐ Daftarin filter itu di Security Config
- ☐ Test: akses endpoint Substation TANPA token → harus ditolak (401)
- ☐ Test: akses pakai token dari login → harus berhasil (200)

Selesai fase ini kalau: endpoint Substation lu udah keproteksi, cuma bisa diakses kalau bawa JWT valid.

Ini fase paling gampang bikin stuck. Kalau macet lebih dari 2-3 jam di satu step, gapapa banget nanya ke gw — jangan mentok sendirian.

FASE 4 — Role-Based Access (Hari 10-11)

- ☐ Ubah field `role` di User jadi Enum: `TEKNISI`, `SUPERVISOR`, `ADMIN`
- ☐ Tambah `@PreAuthorize("hasRole('ADMIN')")` di endpoint yang cuma boleh admin (misal create Substation)
- ☐ Test pakai 2 user beda role, pastiin yang bukan admin ditolak akses

Selesai fase ini kalau: user TEKNISI ga bisa create Substation, tapi ADMIN bisa.

FASE 5 — Entity Inti: Maintenance Report (Hari 12-15)

Sekarang baru masuk ke bagian utama project-nya, setelah fondasi (auth + role) udah solid.

- ☐ Bikin entity `MaintenanceReport` (id, user, substation, title, description, status, category, incidentDate)
- ☐ Bikin relasi `@ManyToOne` ke User dan Substation
- ☐ Endpoint `POST /api/reports` — role TEKNISI aja yang bisa, otomatis ambil user dari token JWT (bukan dari input manual)
- ☐ Endpoint `GET /api/reports` — kalau role TEKNISI, cuma return laporan miliknya sendiri; kalau SUPERVISOR/ADMIN, return semua
- ☐ Endpoint `GET /api/reports/{id}`
- ☐ Endpoint `PATCH /api/reports/{id}/status` — role SUPERVISOR/ADMIN aja

Selesai fase ini kalau: teknisi bisa bikin laporan, dan cuma bisa liat laporannya sendiri; supervisor bisa liat semua & ubah status.

FASE 6 — Upload Foto (Hari 16-17, opsional tapi recommended)

- ☐ Bikin akun gratis di Cloudinary
- ☐ Bikin entity `ReportAttachment` (id, report, fileUrl)
- ☐ Endpoint `POST /api/reports/{id}/attachments` — upload file, dapet URL balik, simpen ke DB

Kalau ini bikin overwhelmed, skip dulu boleh — project udah cukup solid tanpa fitur ini, bisa ditambah belakangan.

FASE 7 — Rapihin & Deploy (Hari 18-20)

- ☐ Tambah Swagger (`springdoc-openapi-starter-webmvc-ui`) — otomatis dapet dokumentasi API
- ☐ Bikin file `.env.example` isinya template environment variable yang dibutuhkan
- ☐ Daftar akun Railway, connect ke GitHub repo
- ☐ Setup environment variable di Railway (DB URL, JWT secret)
- ☐ Deploy, test endpoint lewat URL publiknya
- ☐ Tulis README pakai template yang udah gw kasih sebelumnya

Selesai fase ini kalau: project lu bisa diakses siapa aja lewat link, ada dokumentasi Swagger, dan README-nya jelas.

Kalau Stuck

Di titik manapun lu macet, patokan gw:

- **Stuck < 30 menit** → coba googling error message-nya persis, biasanya ada di StackOverflow
- **Stuck 30 menit - 2 jam** → coba pendekatan lain / baca ulang konsepnya dari awal
- **Stuck > 2 jam atau bingung total soal konsepnya** → langsung tanya ke gw, jangan dipendem. Kirim error message atau kode yang bikin bingung, kita bedah bareng.

Progress pelan tapi konsisten jauh lebih baik daripada ngebut terus stuck total dan project-nya mangkrak.